Laboratorijski delovni list 1

Model peščene sipine: samoorganizirana kritičnost

Univerza v Mariboru · Fakulteta za naravoslovje in matematiko · Oddelek za fiziko

Programsko okolje: Python

Vse naloge rešujete **izključno v Pythonu** (virtualno okolje iz vaje 00).

- `numpy`: `np.zeros`, `np.sum`, `np.unique`, `np.log10`, `np.polyfit`
- `matplotlib.pyplot`: `imshow`, `plot`, `scatter`, `set_xscale`, `set_yscale`
- `scipy.optimize`: `curve_fit` (neobvezno, za nelinearno prilagajanje)
- standardna knjižnica: `time.time`, `sys.setrecursionlimit`

Rešitev oddate kot Jupyter zvezek `.ipynb` z izvedenimi celicami.

1 Teoretične osnove in motivacija

Številni pojavi v naravi potekajo v obliki nenadnih dogodkov, ki nimajo značilne velikosti: potresi, snežni plazovi, gozdni požari, izbruhi na Soncu. Majhnih dogodkov je veliko, velikih malo, pogostost pa z velikostjo pada *potenčno*. Bak, Tang in Wiesenfeld (1987) so predlagali, da se taki sistemi sami od sebe razvijejo v kritično stanje. Temu pravimo **samoorganizirana kritičnost** (SOC). Klasičen zgled je peščeni kup (model BTW).

Opis modela

Na ravno podlago enakomerno pada pesek. Pesek se kopiči in se ob preveliki strmini prerazporedi na sosednja polja. Zrna, ki prečkajo rob, zapustijo sistem. Zanima nas dinamika nastale sipine: kako veliki so plazovi, ali so vsi enako veliki in kako si sledijo v času.

Mreža in dodajanje peska

Podlago predstavimo s celoštevilsko matriko S velikosti $N_x \times N_y$. Element $S_{i,j}$ pove, koliko zrn (oz. kakšna *strmina*) je na polju (i, j). V vsaki iteraciji na središčno polje (c_x, c_y) pade n zrn:

$$S(t+1) = S(t) + F, \qquad F_{i,j} = \begin{cases} n, & (i,j) = (c_x, c_y) \\ 0, & \text{sicer.} \end{cases} \tag{1}$$

Pravilo prerazporeditve

Če strmina polja preseže kritično vrednost S_c, se polje *zruši*:

$$S_{i,j} > S_c: \qquad S_{i,j} \to S_{i,j} - 4, \qquad S_{i\pm1,j} \to S_{i\pm1,j} + 1, \qquad S_{i,j\pm1} \to S_{i,j\pm1} + 1. \tag{2}$$

Sosed, ki zaradi tega preseže S_c, se zruši naprej. Tako nastane **plaz**. Njegova **velikost** s je število rušenj, ki jih sproži eno dodajanje peska. Robovi so odprti: zrno, ki bi padlo izven mreže, se izgubi. To je edini način, da sistem izgublja pesek.

Končno stanje po plazu ni odvisno od vrstnega reda rušenj (t. i. Abelova lastnost modela). Zato lahko plaz preprosto izračunamo z **rekurzivno funkcijo**, ki po vsakem rušenju pokliče samo sebe za vsa štiri sosednja polja.

Stacionarno stanje

Na začetku je podlaga prazna in pesek se le kopiči (prehodni pojav). Ko sipina doseže robove, se vzpostavi **stacionarno stanje**: v povprečju toliko zrn, kot jih dodamo, zapusti sistem čez rob, skupno število zrn pa le še niha okoli povprečja.

Porazdelitev velikosti plazov

Naj bo $N(s)$ število plazov velikosti s. Primerjali bomo dve hipotezi:

$$\text{eksponentna:} \quad N(s) = A\, e^{-Bs} \quad \Rightarrow \quad \ln N = \ln A - B\, s, \tag{3}$$

$$\text{potenčna:} \quad N(s) = A\, s^{-\tau} \quad \Rightarrow \quad \log N = \log A - \tau \log s. \tag{4}$$

Eksponentna porazdelitev ima značilno velikost $1/B$. Potenčna je nima, kar je podpis kritičnega stanja. Na grafu z logaritemsko osjo y je eksponentna funkcija premica, na grafu z logaritemskima osema x in y pa potenčna. Zaradi končne velikosti mreže ima porazdelitev pri velikih s odrez $s_{\max}$.

Kakovost prilagoditve ocenimo s koeficientom determinacije

$$R^2 = 1 - \frac{\sum_k (y_k - \hat{y}_k)^2}{\sum_k (y_k - \bar{y})^2}, \tag{5}$$

šum v časovnih vrstah pa zgladimo z **drsečim povprečjem** širine w: $\bar{y}_k = \frac{1}{w} \sum_{m=k}^{k+w-1} y_m$.

2 Pripravljalna naloga (analitični del)

P1. Ročni izračun plazu. Mreža 3×3 z odprtimi robovi in $S_c = 4$ je v stanju

$$S = \begin{pmatrix} 2 & 4 & 2 \\ 4 & 4 & 4 \\ 2 & 4 & 2 \end{pmatrix}. \tag{6}$$

Na središčno polje dodamo eno zrno. Izvedite vsa rušenja.

- (a) Končno stanje S_{konec}:

- (b) Velikost plazu $s =$ _____
- (c) Število zrn, ki so zapustila mrežo: _____
- (d) Preverite ohranitev peska $\sum S_{\text{prej}} + 1 = \sum S_{\text{konec}} + N_{\text{izgub}}$: _____

P2. Linearizacija. Izpeljite enačbi za eksponentno in potenčno porazdelitev. Katero količino nanesete na os x in katero na os y, da dobite premico? Kako iz naklona k in odseka n premice dobite A, B oz. τ?

eksponentna: _____ potenčna: _____

P3. Eksponent. V simulaciji ste našteli $N(10) = 2000$ in $N(100) = 126$ plazov. Ob predpostavki potenčne porazdelitve izračunajte τ.

$\tau =$ _____

P4. Ocena prehodnega pojava. Največ koliko zrn lahko vsebuje stabilna mreža $N \times N$ s kritično vrednostjo S_c? Recimo, da je v stacionarnem stanju povprečna višina $\langle S \rangle \approx 3{,}2$. Približno koliko iteracij z $n = 1$ potrebuje mreža 31×31, da doseže stacionarno stanje?

$N_{\max} =$ _____ $T_{\text{stac}} \approx$ _____

3 Laboratorijske naloge na računalniku (Python)

Repozitorij

Začetna predloga je v repozitoriju predmeta: `01_vaje/vaje_01.ipynb`. V razdelku *Funkcije* dopolnite telesa funkcij z oznako `TODO`, v razdelku *Testno območje* pa preverite rezultat naloge P1.

Naloga 1: Implementacija modela (25 min)

1. Dopolnite funkciji `init_S` (prazna mreža) in `init_F` (matrika dodajanja).
2. Dopolnite rekurzivno funkcijo `plaz(i, j, S, S_c, stanje)`, ki zruši polje (i, j), če $S_{i,j} > S_c$, in pokliče samo sebe za vse sosede. V slovarju `stanje` šteje rušenja (`"s"`) in izgubljena zrna (`"izguba"`).
3. Na primeru iz P1 primerjajte numerični rezultat z ročnim.

```
def plaz(i, j, S, S_c, stanje):
    if S[i, j] > S_c:
        stanje["s"] += 1
        S[i, j] -= 4
        # TODO: za vsakega od štirih sosedov dodaj zrno (ali ga izgubi
        # čez rob) in rekurzivno pokliči plaz(...)
```

Pri večjih mrežah je globina rekurzije lahko večja od privzete meje (1000 klicev), zato jo povečajte z `sys.setrecursionlimit(10_000)`.

Količina	Ročno (P1)	Numerično
$\sum S_{\text{konec}}$	_____	_____
Velikost plazu s	_____	_____
Izgubljena zrna	_____	_____

Preglednica 1: Preverjanje implementacije na mreži 3×3.

Naloga 2: Prehodni pojav in stacionarno stanje (20 min)

Simulirajte mrežo $N_x = N_y = 31$, $S_c = 4$, $n = 1$ za $T = 30\,000$ iteracij. V vsaki iteraciji shranite velikost plazu (0, če ga ni), skupno število zrn $\sum S$ in število izgubljenih zrn. Vsakih 1000 iteracij izmerite porabljen čas.

- **Slika 1** (trije paneli): (A) $\sum S$ v odvisnosti od iteracije, (B) velikost plazov v odvisnosti od iteracije, (C) končno stanje mreže (`imshow`).
- **Slika 2**: čas izvedbe 1000 iteracij s prikazanima drsečima povprečjema za $w = 5$ in $w = 10$.
- Iz Slike 1A in Slike 2 določite iteracijo T_{stac}, pri kateri se vzpostavi stacionarno stanje, in jo primerjajte z oceno iz P4.

Količina	Vrednost
T_{stac} (ocena P4)	
T_{stac} (simulacija)	
$\langle S \rangle$ v stacionarnem stanju	
Izgubljena zrna na iteracijo ($t > T_{\text{stac}}$)	
Največji plaz $s_{\max}$	

Preglednica 2: Prehod v stacionarno stanje ($N = 31$, $S_c = 4$).

Naloga 3: Porazdelitev velikosti plazov (25 min)

Uporabite le plazove iz stacionarnega stanja ($t > T_{\text{stac}}$). Pogostosti $N(s)$ določite s funkcijo `np.unique` in argumentom `return_counts=True`.

- **Slika 3** (trije paneli): $N(s)$ v (A) lin–lin, (B) lin–log in (C) log–log merilu.
- V vsakem merilu prilagodite premico (`np.polyfit`) na linearizirane podatke in izračunajte R^2. Iz prilagoditev izločite pogostosti $N(s) < 3$ in plazove blizu odreza.
- Določite eksponent τ in ocenite območje s, v katerem potenčni zakon velja.

Model	Linearizacija	Parameter 1	Parameter 2	R^2
linearni	N vs. s	$k =$	$n =$	
eksponentni	$\ln N$ vs. s	$A =$	$B =$	
potenčni	$\log N$ vs. $\log s$	$A =$	$\tau =$	

Preglednica 3: Prilagoditve porazdelitve velikosti plazov.

Naloga 4: Vpliv kritične strmine (20 min)

Simulacijo ponovite na mreži 21×21 s $T = 20\,000$ iteracijami za $S_c \in \{4, 6, 8\}$. Vse tri porazdelitve $N(s)$ narišite na skupen log–log graf (**Slika 4**) in izpolnite preglednico. Ali se porazdelitev plazov spremeni? Razmislite, kaj se zgodi z modelom, če vse višine $S_{i,j}$ zamaknemo za isto konstanto.

S_c	T_{stac}	$\langle S \rangle$	$\langle s \rangle$	$s_{\max}$	τ
4					
6					
8					

Preglednica 4: Vpliv kritične strmine S_c na dinamiko sipine ($N = 21$).

4 Vprašanja za razpravo in zaključni zagovor

Razmislek 1: Kritičnost brez nastavljanja

Pri feromagnetu moramo temperaturo natančno nastaviti na T_c, da opazimo potenčne zakone. Katerega parametra v modelu sipine ni treba nastavljati? Zakaj je pomembno, da pesek dodajamo počasi, torej da se plaz konča pred naslednjim dodajanjem?

Razmislek 2: Rekurzija in njene meje

Kako se največja globina rekurzije spreminja z velikostjo mreže? Kaj bi se zgodilo pri $N = 101$ brez povečane meje? Opišite iterativno izvedbo s seznamom (skladom) polj, ki jih je treba preveriti. Zakaj je zaradi Abelove lastnosti rezultat enak?

Razmislek 3: Končna velikost sistema

Kako je odrez porazdelitve $s_{\max}$ povezan z velikostjo mreže? Zakaj bi morali pri strogi analizi primerjati porazdelitve za več velikosti N? Poskusi z realnim peskom pogosto ne kažejo čistega potenčnega zakona, s kupi riža pa ga (Frette in sod., 1996). Kaj bi lahko bil razlog?

Razmislek 4: Stohastično pravilo (domača naloga)

Spremenite model tako, da preobremenjeno polje odda le eno zrno naključno izbranemu sosedu. Kako se spremeni porazdelitev velikosti plazov? Ali je sistem še vedno kritičen?